

# **Python Language Programming**

**Conditions, Loops**

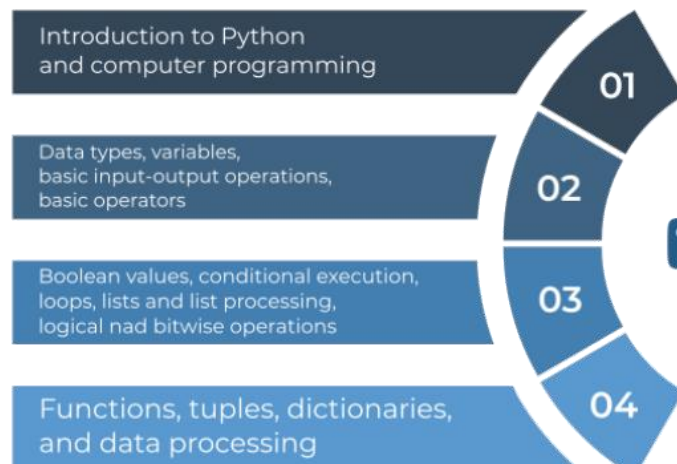
- **Disclaimer**

- These slides and notes are based on the contents of the courses:

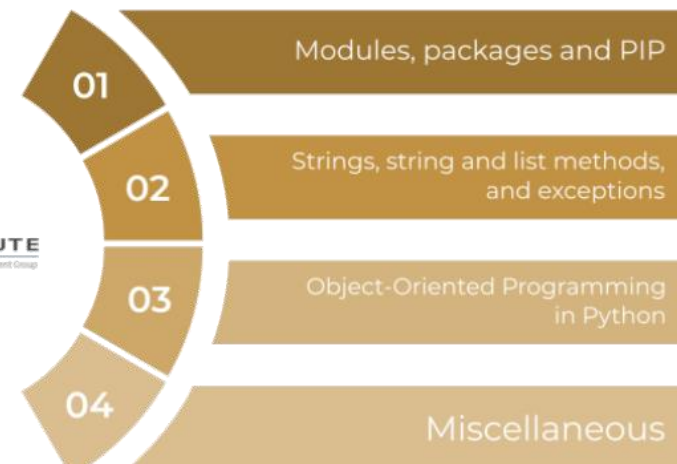
- **Python Essentials 1 and 2**

- aligned with the **PCEP** and **PCAP** certifications, respectively;
- developed by the OpenEDG Python Institute;  
([www.pythoninstitute.org](http://www.pythoninstitute.org))
- offered in the Cisco Networking Academy program.

Python Essentials 1



Python Essentials 2

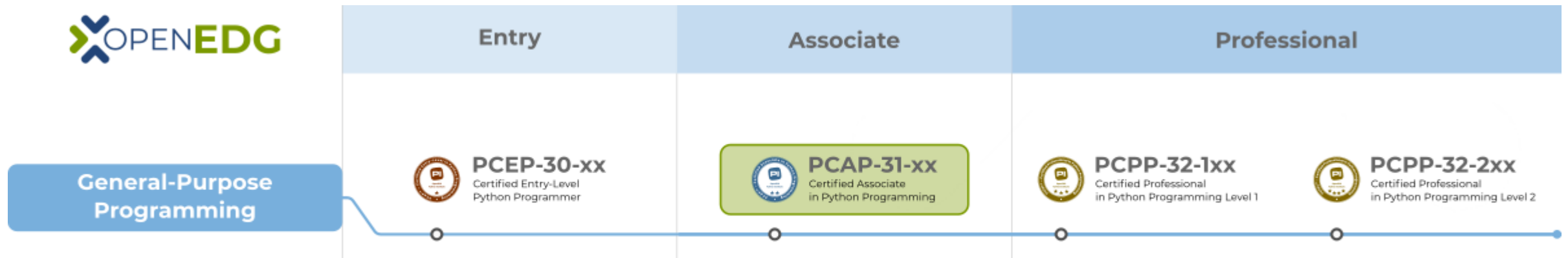


- **Disclaimer**

- These slides and notes are based on the contents of the courses:

- **Python Essentials 1 and 2**

- aligned with the **PCEP** and **PCAP** certifications, respectively;
- developed by the OpenEDG Python Institute;  
([www.pythoninstitute.org](http://www.pythoninstitute.org))
- offered in the Cisco Networking Academy program.



# Table of Contents

- **Conditional Execution**
  - **if statement**
    - if-else statement
    - elif statement
- **Loops**
  - **while**
    - else branch
  - **for**
    - else branch
    - iterators (range)
  - **break and continue statements**
- **Bibliography**

# Python Language Programming

- **Conditional Execution**

- **`if` statement**

- This conditional statement consists of the following elements:

- the `if` keyword;
      - a logic expression;
      - a colon followed by a newline;
      - an indented instruction or set of instructions.

```
if true_or_not:  
    do_this_if_true
```

**Indentation:**

- the indentation may be by inserting:
  - a number of spaces (recommended to use four spaces of indentation)
  - a tab character;
- if there is more than one instruction in the indented part, the indentation should be the same in all lines.

# Python Language Programming

- Conditional Execution
  - **if-else statement**

```
if the_weather_is_good:
    go_for_a_walk()
    have_fun()
else:
    go_to_a_theater()
    enjoy_the_movie()
have_lunch()
```

```
if the_weather_is_good:
    if nice_restaurant_is_found:
        have_lunch()
    else:
        eat_a_sandwich()
else:
    if tickets_are_available:
        go_to_the_theater()
    else:
        go_shopping()
```

**Nested if-else statements**

# Python Language Programming

- **Conditional Execution**

- **elif statement**

- used to check more than just one condition, and to stop when the first statement which is true is found

```
if condition1:
    instruction1
else:
    if condition2:
        instruction2
    else:
        if condition3:
            instruction3
        else:
            instruction4
```



```
if condition1:
    instruction1
elif condition2:
    instruction2
elif condition3:
    instruction3
else:
    instruction4
```

# Python Language Programming

- **Conditional Execution**

- **elif statement**

- used to check more than just one condition, and to stop when the first statement which is true is found

```
if the_weather_is_good:  
    go_for_a_walk()  
elif tickets_are_available:  
    go_to_the_theater()  
elif table_is_available:  
    go_for_lunch()  
else:  
    play_chess_at_home()
```

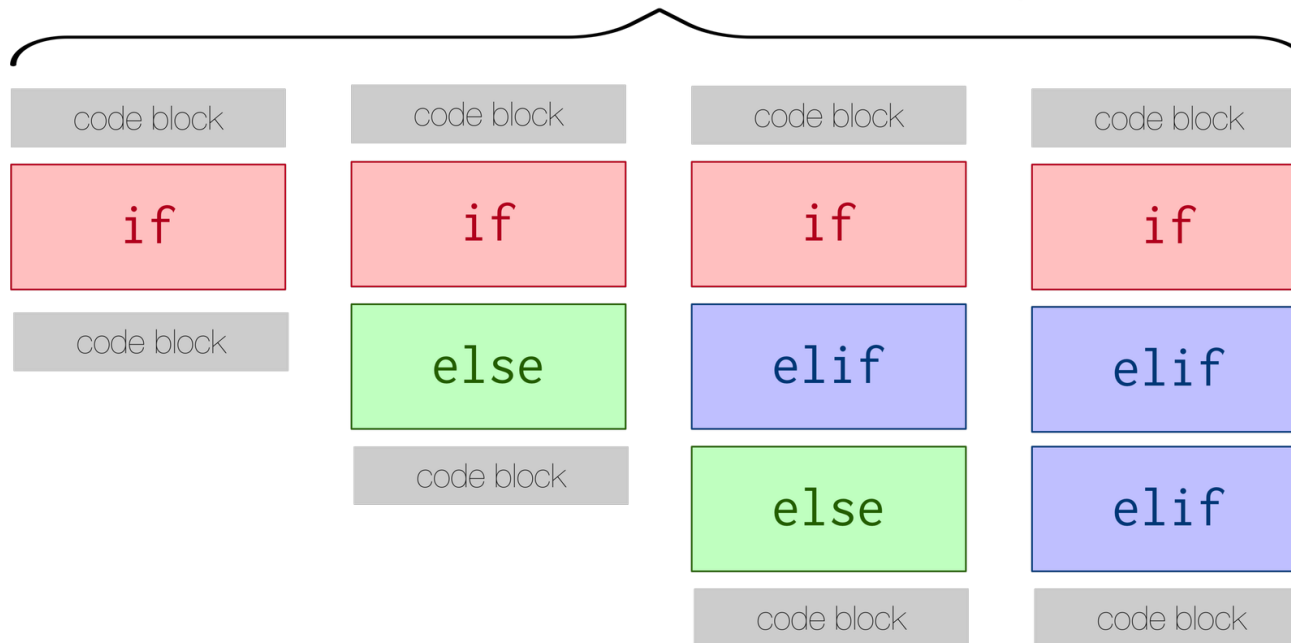
- else is always the last branch of the cascade, regardless of whether you've used `elif` or not;
      - else is an optional part of the cascade, and may be omitted;



# Python Language Programming

- Conditional Execution

valid if/elif/else order examples



# Python Language Programming

- **Loops**
  - **while**

```
while conditional_expression:  
    instruction_one  
    ...  
    instruction_n
```

- syntax is identical to the C equivalent
- when the condition is met, it performs its statements only once
- repeats the execution as long as the condition evaluates to `True`
- Python does not provide a `do...while` loop (contrarily to C language)

# Python Language Programming

- Loops
  - while

```
# Store the current largest number here.
largest_number = -999999999

# Input the first value.
number = int(input("Enter a number or type -1 to stop: "))

# If the number is not equal to -1, continue.
while number != -1:
    # Is number larger than largest_number?
    if number > largest_number:
        # Yes, update largest_number.
        largest_number = number
    # Input the next number.
    number = int(input("Enter a number or type -1 to stop: "))

# Print the largest number.
print("The largest number is:", largest_number)
```

# Python Language Programming

- **Loops**

- **while**

```
odd_numbers = 0
even_numbers = 0

# Read the first number.
number = int(input("Enter a number or type 0 to stop: "))

while number != 0: # 0 terminates execution.
    if number % 2 == 1: # Check if the number is odd.
        # Increase the odd_numbers counter.
        odd_numbers += 1
    else:
        # Increase the even_numbers counter.
        even_numbers += 1
    # Read the next number.
    number = int(input("Enter a number or type 0 to stop: "))

print("Odd numbers count:", odd_numbers)
print("Even numbers count:", even_numbers)
```

A program that reads a sequence of numbers and counts how many numbers are even and how many are odd. The program terminates when zero is entered.

# Python Language Programming

- **Loops**

- **while**

- **else branch**

- the `while` loop supports `else` branch on exit

- `else` branch is always executed once, regardless of whether the loop has entered its body or not.

```
while conditional_expression:
    instruction_one
    ...
    instruction_n
else:
    instruction_one
    ...
    instruction_n
```

# Python Language Programming

- Loops
  - for

```
for variable in sequence:  
    instruction_one  
    ...  
    instruction_n
```

- syntax is different from the C equivalent
- a variable iterates on the elements of a sequence
- a sequence can be a lot of things:
  - a list, dictionary or set
  - a string
  - an **iterator**

# Python Language Programming

- **Loops**

- **for**

- the `for` loop supports `else` branch on exit
    - `else` branch is always executed once, regardless of whether the loop has entered its body or not.

```
for variable in sequence:
    instruction_one
    ...
    instruction_n
else:
    instruction_one
    ...
    instruction_n
```

# Python Language Programming

- Loops

- for

- **range** iterator

```
for variable in range(start, end, step):  
    instruction
```

- Iterators are objects that return a sequence of other objects one element at a time using the method `next()`
      - The classical iterator is `range(start, end, step)`:
        - **start**: optional parameter specifying the starting number of the sequence (**0 by default**)
        - **end**: optional parameter specifying the end of the sequence generated (it is **NOT included!!!**),
        - **step**: optional parameter specifying the iteration in the sequence (**1 by default**). 'Step' requires that 'Start' and 'Stop' are inserted

If Start > End the loop is not executed, unless Step is < 0



# Python Language Programming

- Loops

- for

- **range** iterator

- There are three ways to call the range function:
        - With one integer argument it counts from 0 to stop
          - `range(stop)`
        - With two integer arguments it counts from start to stop
          - `range(start, stop)`
        - With three integer arguments it counts from start to stop, # with a defined step size
          - `range(start, stop, step)`

# Python Language Programming

- Loops

- break and continue statements

- **break** - exits the loop immediately, and unconditionally ends the loop's operation; the program begins to execute the nearest instruction after the loop's body;
    - **continue** - behaves as if the program has suddenly reached the end of the body; the next turn is started and the condition expression is tested immediately.

## # break - example

```
print("The break instruction:")
for i in range(1, 6):
    if i == 3:
        break
    print("Inside the loop.", i)
print("Outside the loop.")
```

## # continue - example

```
print("\nThe continue instruction:")
for i in range(1, 6):
    if i == 3:
        continue
    print("Inside the loop.", i)
print("Outside the loop.")
```

# Bibliography

- Burkov, A. (2019). The hundred-page machine learning book. Andriy Burkov.
- **Cisco. (2022). Python Essentials. Retrieved September 4, 2024, from <https://www.netacad.com/>**
- Geron, A. (2019). Hands-on machine learning with scikit-learn, keras, and tensorflow. O'Reilly.
- Ekman, M. (2022). Learning Deep Learning. Addison-Wesley.
- Mckinney, W. (2017). Python for data analysis: Data wrangling with pandas, numpy, and ipython. O'Reilly.
- Raschka, S., & Mirjalili, V. (2019). Python machine learning: Machine learning and deep learning with python, scikit-learn, and tensorflow. Packt Publishing.
- Russel, S., & Norvig, P. (2018). Artificial intelligence: A modern approach. Pearson Education Limited.